

Software Engineering

Urheberrechtlich geschütztes Skript

Rechtsvermerk:

Dieses Dokument ist urheberrechtlich geschützt und beinhaltet Informationen des aus dem Unternehmen des Lehrenden. Das Dokument ist als begleitendes Lehrmaterial für die Kursteilnehmer (Studierenden) an einer Hochschule freigegeben.

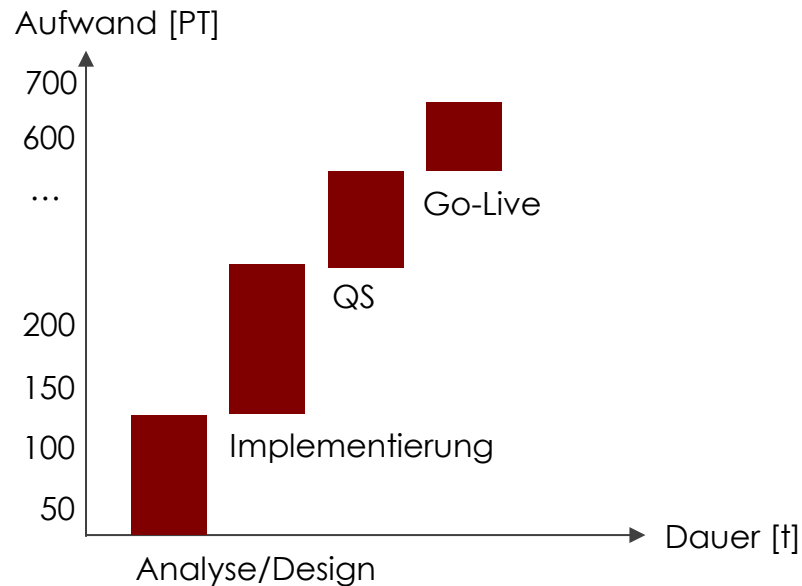
Das Dokument darf im Sinne der einfachen Nutzungsrechte lediglich für die private Weiterbildung und als Lernunterlage zur Prüfungsvorbereitung verwendet werden.

Eine Verwertung, Vervielfältigung, Veröffentlichung oder Weitergabe – auch in Auszügen – ist nicht erlaubt und bedarf der ausdrücklichen Zustimmung des Urhebers.

Die Angabe/Verwendung der Quellen je Folie sind im Original PowerPoint-Dokument dieses Skripts im Notizenbereich vermerkt. In der PDF-Ausfertigung dieses Skripts sind die Quellen je Folie nicht ablesbar. Auf Anfrage werden die Quellen bereitgestellt.

Ressourcenplanung

Aufwand in [PT]	Analyse/Design	Realisierung	QS	Go-Live	Kummuliert
Projektleitung	20	20	20	20	80
Consulting	50	30	40	10	130
SW-Architekt	40	40	20	10	110
SW-Entwickler	10	200	75	25	310
Sicherheitsbeauftragter	5	5	5	5	20
QS-Manager	5	5	30	5	45
Summe	130	300	190	75	695



Welche Sichten fehlen?

Kapazitätsplanung

Annahme:

Sie planen die Realisierungsphase eines Softwareprojekts im agilen Vorgehen.

Die reinen Entwicklungsaufwände belaufen sich auf **50 Personentage (PT)** und lassen sich überschneidungsfrei parallelisieren.

Die **Sprintlänge** soll **zwei Wochen (10AT)** dauern. Ihnen stehen fünf Entwickler mit je **0,5 FTE** zur Verfügung.

Frage:

Wie viele Sprints brauchen Sie?

Vertragliche Auswirkung der Vorgehensmodelle

- Die Festlegung auf ein bestimmtes Projektvorgehensmodells muss gut durchdacht sein, da beide Parteien vertragliche verpflichtet sind ihren Aufgaben nachzukommen
- Klassische/Konventionelle Vorgehensmodelle erfordern
 - eine hohe Disziplin bei der Einhaltung der formalen Regeln und Prozesse im Rahmen der Projektdurchführung sowie ein klar umrissenes Rollenverständnis
 - gute Vorarbeiten durch den Fachbereich im Sinne der **Scopedefinition**
 - Früh festgelegte Anforderungen und freigegebene/unveränderbare **Budgets**
- Agile Vorgehensmodelle erfordern
 - eine hohe Mitwirkungspflicht des Kunden (respektive **Product Owner**)
 - eine stringente **Zeit-** und **Ressourcenplanung** im Sinne der zugrundeliegenden **Sprints**
 - Ein klares Projektziel, damit die erhöhte Flexibilität nicht zum Scheitern führt

Vertragsgestaltung – allgemeine Gedanken

- Die soeben genannten Problemfelder wirken sich ebenfalls auf die Vertragsgestaltung aus
- Es muss zwischen einem Festpreisprojekt (Werkvertrag) und einer Beauftragung nach Time & Material entschieden werden
- Die „gelebte“ Zusammenarbeit zwischen den Vertragsparteien spielt eine wesentliche Rolle (harmonisch/langjährig erfolgreich oder erzwungen)
- In Festpreisprojekten liegt das Risiko tendenziell beim Auftragnehmer, bei T&M Projekten steuert der Auftraggeber
- Insbesondere Werkverträge erfordern eine detaillierte Ausgestaltung
 - verbindliche Meilensteine und Liefertermine (Time)
 - einen vereinbarten Budgetumfang (Budget)
 - Qualitätsanforderungen im Rahmen eines Testkonzepts (Quality)
 - einen festgelegten Projektumfang im Sinne der geforderten Software und dazugehörigen Dokumentation sowie weiterer Ergebnisobjekte (Scope)
 - Konsequenzen bei Nichteinhaltung o.g. Parameter und Mitwirkungspflichten des Auftraggebers

Vertragsarten und Auswirkungen auf das Projekt – Auszug

	Festpreisverträge	Time & Material
Time	Meilensteine werden vertraglich fixiert, bei Nichteinhaltung droht Pönalezahlung	(Grobe) Zeitplanung im Sinne des Time-to-Market wird vorgegeben
Budget	Budgetfestlegung vor Projektbeginn (fix)	Budgetfestlegung zur Laufzeit variabel
Budget	Vertragliche ausgewiesene Risikoaufschläge üblich/ratsam	Keine Risikoaufschläge notwendig
Quality	Kostenneutrale Gewährleistung ist Vertragsbestandteil und Teil des Testkonzepts	Korrekturmaßnahmen werden bezahlt
Scope	Aufwandsschätzung und Bepreisung entscheidet über kommerziellen Erfolg	Priorisierung und sinnvolle Schneidung der Projektinhalte wichtig
Scope	Anforderungen müssen im Vorfeld eindeutig definiert sein	Anforderungen meist noch instabil, Konkretisierung erfolgt im Projektverlauf
Scope	Es gibt i.d.R. ein festgelegtes Delivery Modell, in dem vorab definiert wird, welche Ergebnisobjekte geliefert werden sollen	Das Delivery Modell orientiert sich oftmals an den Mindestvorgaben, da jegliche Art der Dokumentation bezahlt werden muss
...	...	...

Vertragsgestaltung – Herausforderungen in der Praxis

- Die Anforderungen stehen zum Zeitpunkt der Vertragsgestaltung oft noch nicht (final) fest, dadurch ist
 - der Projektumfang/die Komplexität nur schwer einzuschätzen (Scope)
 - eine verlässliche Aufwandsschätzung nicht bzw. nur schwer möglich (Budget)
 - die Projektlaufzeit nur schwer zu terminieren (Time)
 - eine gezielte Ressourcen- und Einsatzplanung unmöglich (Budget/Quality)
- Häufig kann zu Projektbeginn nur eine Kostenindikation angegeben werden, bestenfalls sollte hier nur ein Aufwandskorridor genannt werden
- Unvorhergesehen Änderungen bzw. unspezifizierte Anforderungen lassen sich am einfachsten durch transparente Risikoaufschläge puffern
- Besonders bei Werkverträgen tragen die o.g. Punkte zu einem kommerziellen Risiko bei, ein mögliches Szenario ist die Analyse/Designphase als T&M, die Realisierung und QS-Phase als Festpreis abzuwickeln

Software Engineering

- Requirements Engineering
- Anforderungsanalyse
- Systemkontextanalyse
- Anforderungsdokumentation und –modellierung (EPK/UML)
- Kano Modell
- Aufwandsschätzung/-verfahren

Requirements Engineering im Software Lebenszyklus

Vorgehensmodell (klassisch/agil)

Projekt-/Anforderungs-/Qualitäts-/Konfigurations-Risiko-...Management

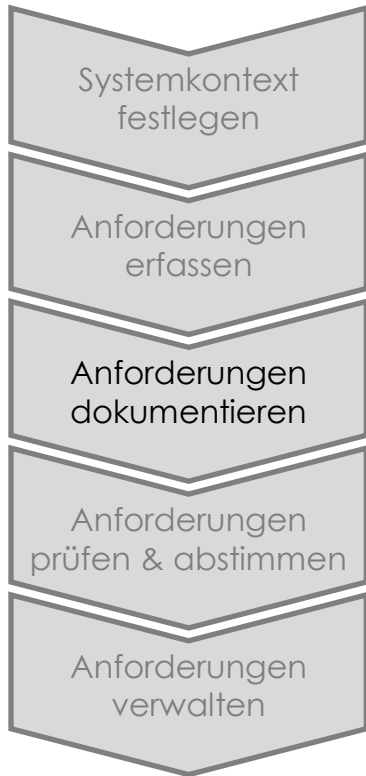
Analysephase	Systementwurf	Realisierung	Test	Abnahme	Inbetriebnahme
- Systemvision - Kontextanalyse - Ermittlung - Dokumentat. - Spezifikation - Modellierung - Validierung	- Grobentwurf - Feinentwurf	- Implementierung - Integration - Migration	- Unit-Test - Komponententests - Integrationstests	- Abnahmetests - Externe QS - Fachliche Abnahme	- Hypercare - Änderungen - Fehlerbehebung - Wartung

Aufwands-schätzung

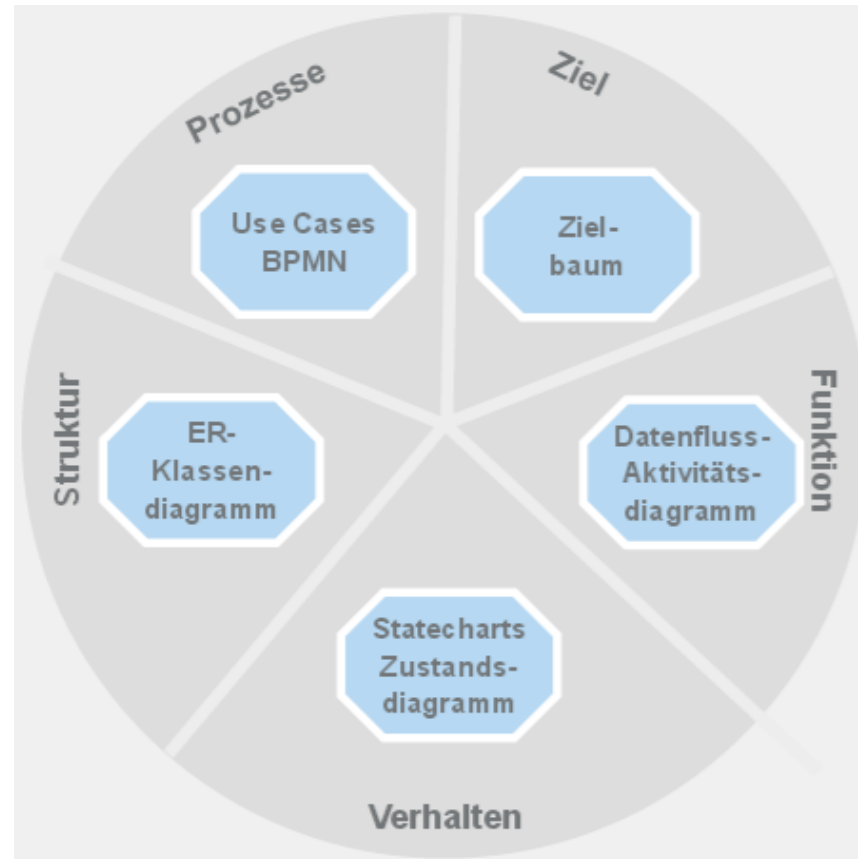
Aufgabenpakete/Ergebnisobjekte

- Anforderungsanalyse - Systemkontextdiagramm - Ereignisgesteuerte Prozessketten - Fachklassenmodell - Maskendesign/ Maskenablaufplan - Validierungsregeln/ Maskenlogik - Navigationskonzept/ Styleguide - Anwendungsfälle - Sicherheitskonzept	- Schnittstellendokumentation - Systemarchitektur - Softwarearchitektur - Datenmodell - Sizing Modell	- Unit-Tests/ Modultests (technisch) - Smoketests (technisch) - Last- & Performancetests (technisch) - Testfälle/ Testnachweis der Realisierung (fachlich) - Regressionstests (fachlich) - Akzeptanztests (→ fachl. Abnahme)	- Wartungskonzept - Betriebshandbuch - Monitoringkonzept - SLA-Konzept - ITIL-Prozesse
---	---	---	--

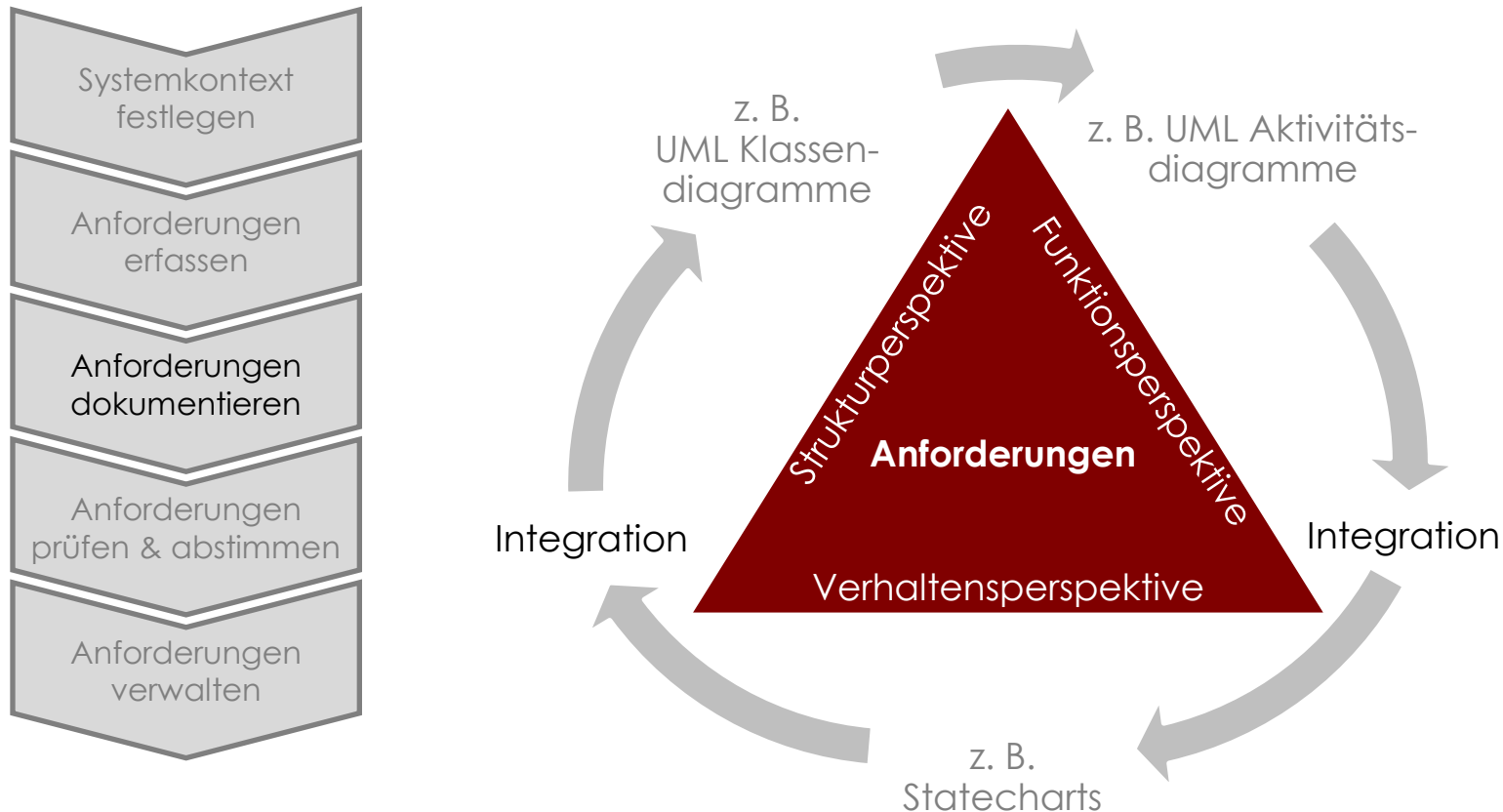
Modellierung



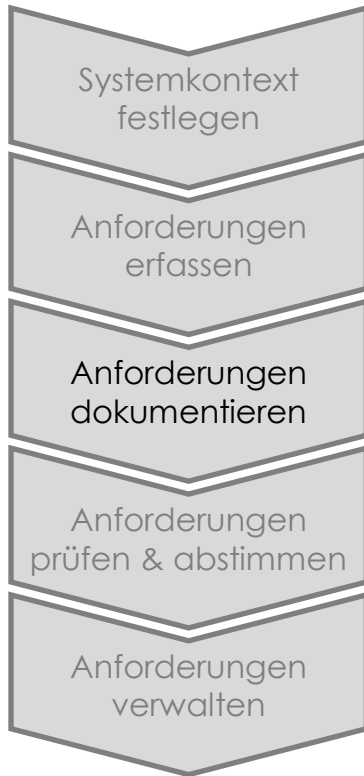
Modellierungssprachen | Modellierungssichten



Die drei Perspektiven der Anforderungsdokumentation - technisch



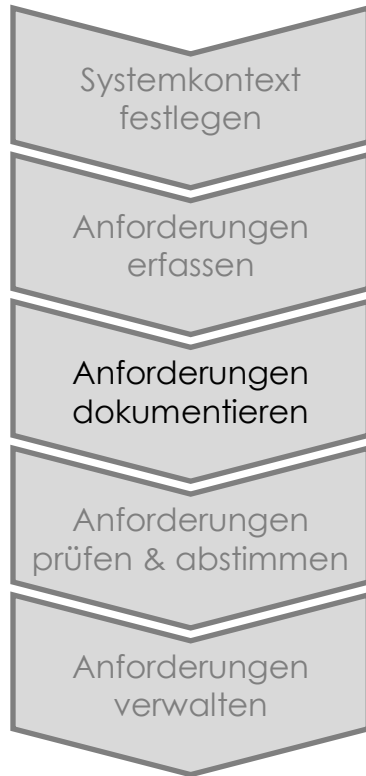
Die drei Perspektiven der Anforderungsdokumentation - technisch



Strukturperspektive (Datenperspektive)

- Es wird eine statisch-strukturelle Perspektive auf die Anforderungen eingenommen (Datentypen, deren Attribute sowie Beziehungen zwischen den Datentypen)
- Es wird die Struktur der Ein- und Ausgabedaten sowie statisch-strukturelle Aspekte von Nutzung und Abhängigkeitsbeziehungen des Systems im Systemkontext dokumentiert
- Beispiel: Die zu nutzenden Dienste eines externen Systems

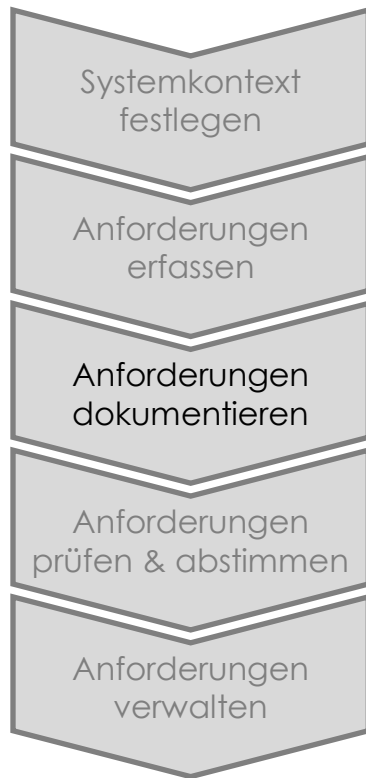
Die drei Perspektiven der Anforderungsdokumentation - technisch



Funktionsperspektive

- Es wird dokumentiert, welche Informationen bzw. Daten aus dem Systemkontext durch das zu entwickelnde System bzw. dessen Funktionen
 - manipuliert werden
 - in den Systemkontext fließen
- Nach Bedarf wird auch die Systematik der Funktionsführung zur Verarbeitung der Eingabedaten dokumentiert
- Die Funktionsperspektive fokussiert sich demnach auf die Transformation von Eingabedaten aus der Umgebung des Systems in Ausgabedaten in die Umgebung

Die drei Perspektiven der Anforderungsdokumentation - technisch



Verhaltensperspektive

- Dokumentation der Einbettung des Systems in den Systemkontext (zustandsorientiert)
- Es wird z. B. die Reaktion des Systems auf Ereignisse im Systemkontext, Bedingungen eines Zustandswechsels sowie Effekte definiert (bspw. wenn das betrachtete System für ein weiteres System ein Ereignis darstellt, externe Stimulation)
- Die Definition eines endlichen Automaten umfasst eine Menge von Zuständen und Zustandsübergängen, die, abhängig vom aktuellen Zustand des Automaten, durch das Eintreten eines Ereignisses ausgeführt werden

Unified Modeling Language

Unified Modeling Language

- Sprache zur
 - Modellierung
 - Dokumentation
 - Spezifizierung
 - Visualisierungkomplexer Softwaresysteme
- In den 90ern von der Object Management Group entwickelt – mittlerweile ISO standardisiert
- Dominierende Sprache (de facto Standard) in der SE
- Besteht aus einer Sammlung vorwiegend grafischer Sprachen zur Erstellung von Anforderungs- und Entwurfsmodellen aus verschiedenen Perspektiven

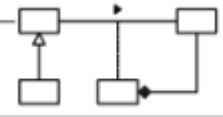
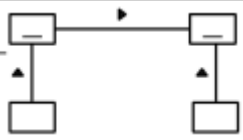
UML – Übersicht der Diagramme

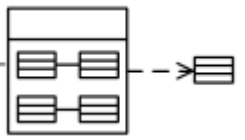
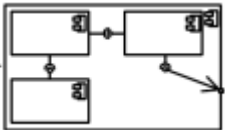
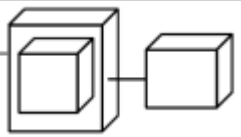
Diagramme der UML

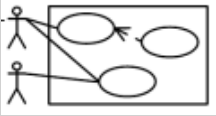
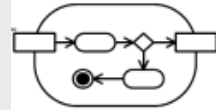
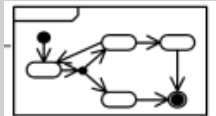
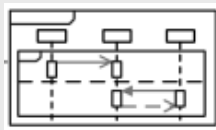
Strukturdiagramme	Verhaltensdiagramme	
		Interaktionsdiagramme
Klassendiagramm		
Paketdiagramm	Use-Case-Diagramm	Sequenzdiagramm
Objektdiagramm	Aktivitätsdiagramm	Kommunikationsdiagramm
Kompositionsstrukturdiagramm	Zustandsautomat	Timingdiagramm
Komponentendiagramm		Interaktionsübersichtsdiagramm
Verteilungsdiagramm		

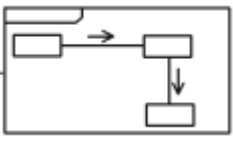
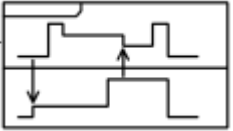
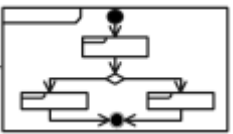
■ 13 Diagramme

- 6 Strukturdiagramme
- 7 Verhaltensdiagramme
 - davon 4 Interaktionsdiagramme

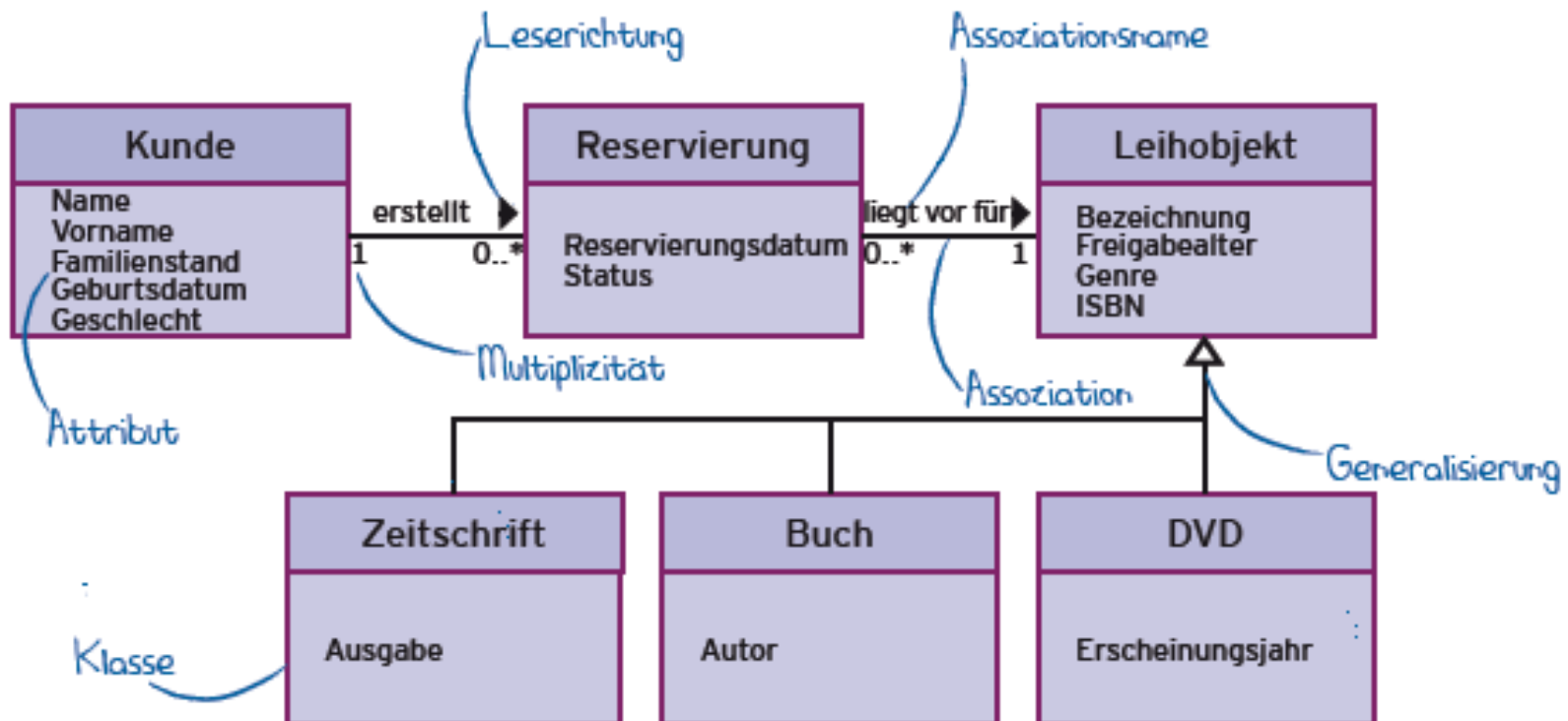
Diagrammtyp	Diese zentrale Frage beantwortet das Diagramm	Stärken
Klassendiagramm 	Aus welchen Klassen besteht mein System und wie stehen diese untereinander in Beziehung?	▪ Beschreibt die statische Struktur des zu entwerfenden oder abzubildenden Systems ▪ Enthält alle relevanten Strukturzusammenhänge und Datentypen ▪ Bildet die Brücke zu den dynamischen Diagrammen
Paketdiagramm 	Wie kann ich mein Modell so schneiden, dass ich den Überblick bewahre?	▪ Organisiert das Systemmodell in größere Einheiten durch logische Zusammenfassung von Modellelementen ▪ Modellierung von Abhängigkeiten
Objektdiagramm 	Welche innere Struktur besitzt mein System zu einem bestimmten Zeitpunkt zur Laufzeit (Klassendiagrammschnappschuss)	▪ Zeigt Objekte und deren Attributsbelegungen zu einem bestimmten Zeitpunkt ▪ Wird nur beispielhaft zur Veranschaulichung verwendet ▪ Detailniveau wie im Klassendiagramm ▪ Sehr gute Darstellung von Mengenverhältnissen

Diagrammtyp	Diese zentrale Frage beantwortet das Diagramm	Stärken
Kompositionsstrukturdiagramm 	Wie sieht das Innenleben einer Klasse, einer Komponente, eines Systemteils aus?	▪ Ideal für die Top-down-Modellierung des Systems ▪ Mittels Detailniveau, zeigt Teile eines „Gesamtelements“ und deren Mengenverhältnisse
Komponentendiagramm 	Wie werden meine Klassen zu wieder verwendbaren, verwaltbaren Komponenten zusammengefasst und wie stehen diese miteinander in Beziehung?	▪ Zeigt Organisation und Abhängigkeiten einzelner technischer Systemkomponenten ▪ Modellierung angebotener und benötigter Schnittstellen möglich
Verteilungsdiagramm 	Wie sieht das Einsatzumfeld (Hardware, Server, Datenbanken, ...) des Systems aus? Wie werden die Komponenten zur Laufzeit wohin verteilt?	▪ Zeigt das Laufzeitumfeld des Systems mit den „greifbaren“ Systemteilen (meist Hardware) ▪ Hohes Abstraktionsniveau, kaum Notationselemente

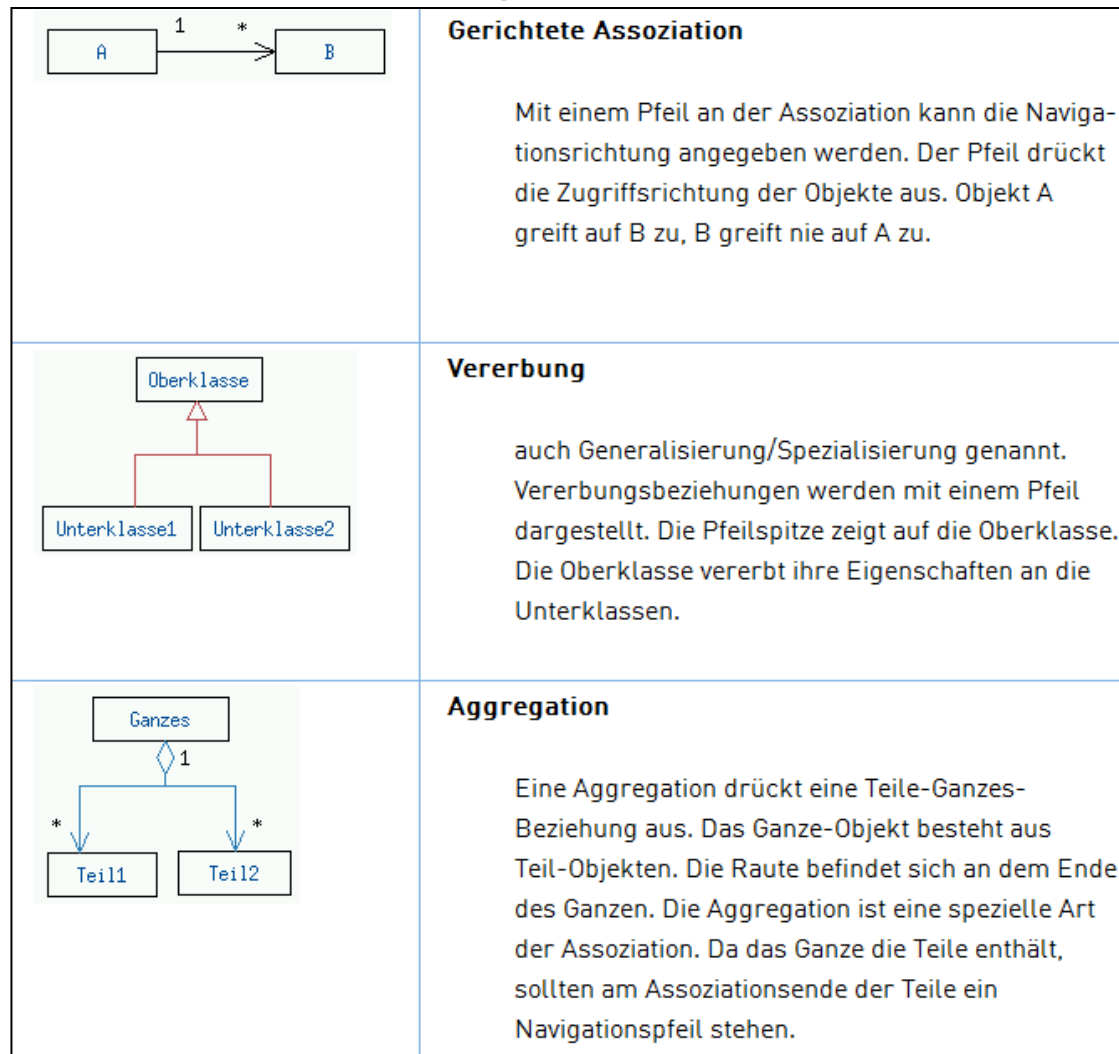
Diagrammtyp	Diese zentrale Frage beantwortet das Diagramm	Stärken
Use-Case –Diagramm 	Was leistet mein System für seine Umwelt (Nachbarsysteme, Stakeholder) ?	■ Präsentiert die Außensicht auf ein System ■ Geeignet zur Kontextabgrenzung ■ Hohes Abstraktionsniveau, einfache Notationsmittel
Aktivitätsdiagramm 	Wie läuft ein bestimmter flussorientierter Prozess oder ein Algorithmus ab?	■ Sehr detaillierte Visualisierung von Abläufen mit Bedingungen, Schleifen, Verzweigungen, Parallelisierung und Synchronisation möglich
Zustandsautomat 	Welche Zustände kann ein Objekt, eine Schnittstelle, eine Use Case, ... bei welchen Ereignissen annehmen?	■ Präzise Abbildung eines Zustandsmodells mit Zuständen, Ereignissen, Nebenläufigkeiten, Bedingungen, Ein- und Austrittsaktionen ■ Schachtelung möglich
Sequenzdiagramm 	Wer tauscht mit wem welche Informationen in welcher Reihenfolge aus?	■ Stellt den zeitlichen Ablauf des Informationsaustausches zwischen Kommunikationspartnern dar ■ Schachtelung und Flusssteuerung (Bedingungen, Schleifen, Verzweigungen) möglich

Diagrammtyp	Diese zentrale Frage beantwortet das Diagramm	Stärken
Kommunikationsdiagramm 	Wer kommuniziert mit wem? Wer „arbeitet“ im System zusammen?	▪ Stellt den Informationsaustausch zwischen Kommunikationspartnern dar ▪ Überblick steht im Vordergrund (Details und zeitliche Abfolge weniger wichtig)
Timingdiagramm 	Wann befinden sich verschiedene Interaktionspartner in welchem Zustand?	▪ Visualisiert das exakte zeitliche Verhalten von Klassen, Schnittstellen, ... ▪ Geeignet für Detailbetrachtungen, bei denen es überaus wichtig ist, dass ein Ereignis zum richtigen Zeitpunkt eintritt
Interaktionsübersichtsdiagramm 	Wann läuft welche Interaktion ab?	▪ Verbindet Interaktionsdiagramme (Sequenz-, Kommunikations- und Timingdiagramme) auf Top-Level-Ebene ▪ Hohes Abstraktionsniveau ▪ Gut geeignet als Strukturierung der Interaktionsdiagramme

Beispiel Klassendiagramm (Strukturperspektive)



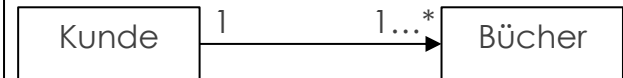
Beispiel Klassendiagramm



Beispiele:

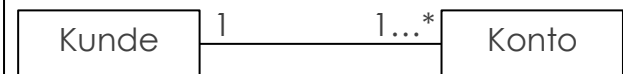
Gerichtete Assoziation:

Ein Kunde kauft ein oder mehrere Bücher. Die Bücher haben keinen Bezug zum Käufer.



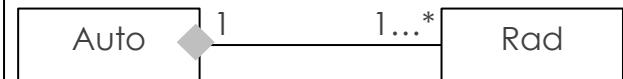
Ungerichtete Assoziation:

Ein Kunde hat ein oder mehrere Konten, ein Konto gehört immer genau einen Kunden.



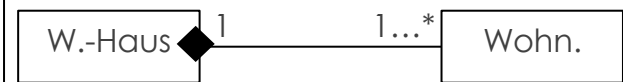
Aggregation:

Ein Rad ist Teil eines Autos. Ein Rad kann problemlos unabhängig vom Auto existieren.

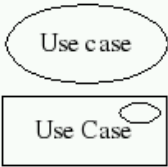


Komposition:

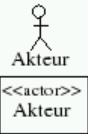
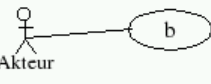
Eine Wohnung ist Teil eines Wohnhauses. Ohne Wohnhaus kann die Wohnung nicht existieren.



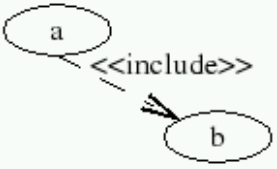
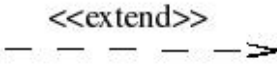
Notation Use Case Diagramm

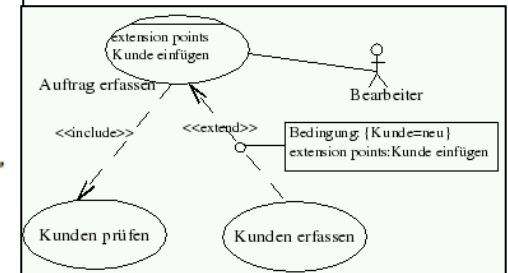
	System Das Rechteck stellt das geplante System dar. Der Name gibt den Namen des Systems an. Ein Use Case Diagramm kann auch mehrere Systeme enthalten. Dadurch kann ein System in Teilsysteme gegliedert werden.
	Use Case (Anwendungsfall) Eine Ellipse stellt einen Anwendungsfall des Systems dar. Ein Anwendungsfall ist ein in sich abgeschlossener Vorgang, der für einen oder mehrere Akteure ein beobachtbares Ergebnis liefert. Er beschreibt aus Sicht der Akteure welche Leistungen das System für den Anwender zur Verfügung stellt. Ein Use Case stellt somit einen Teil der Gesamtfunktionalität des Systems dar. In UML 2.0 kann auch ein Rechteck, das mit einer Ellipse markiert wird, als Use-Case-Symbol verwendet werden. Der Name kann innerhalb oder außerhalb des Symbols stehen.

Notation Use Case Diagramm

	Akteur Ein Akteur ist ein Element, das nicht zum geplanten System gehört. Er kann eine Person sein, die auf das System zugreift, oder ein anderes System, das mit dem geplanten System kommuniziert. Die UML erlaubt mehrere Symbole zur Darstellung eines Akteurs. Er kann als Strichmännchen dargestellt werden. Es ist optional erlaubt ein Klassensymbol zu verwenden, das mit dem Stereotyp <<actor>> markiert wird. Zusätzlich können eigene Symbole verwendet werden um nicht menschliche Akteure darzustellen.
	Assoziation Eine Linie stellt eine Assoziation zwischen einem Akteur und einem Use Case dar. Sie beschreibt den Zugriff des Akteurs auf die Funktionalität, die das System in diesem Use Case zur Verfügung stellt, bzw. eine Antwort des Systems an einen Akteur.

Notation Use Case Diagramm

	include-Assoziation Bei der include Beziehung verwendet ein Use Case die Funktionalität, die ein anderer Use Case zur Verfügung stellt. Der inkludierte Use Case wird immer ausgeführt. Der Use Case a importiert die Funktionalität des Use Case b.
	extend-Assoziation Die extend Beziehung beschreibt die Erweiterung der Funktionalität eines Use Cases durch einen anderen Use Case. Man kann dadurch optionales Verhalten beschreiben, bzw. Funktionen modellieren, die nur unter bestimmten Bedingungen ausgeführt werden.



Aktivitätsdiagramm (Funktionsperspektive)

Wie realisiert mein System ein bestimmtes Verhalten

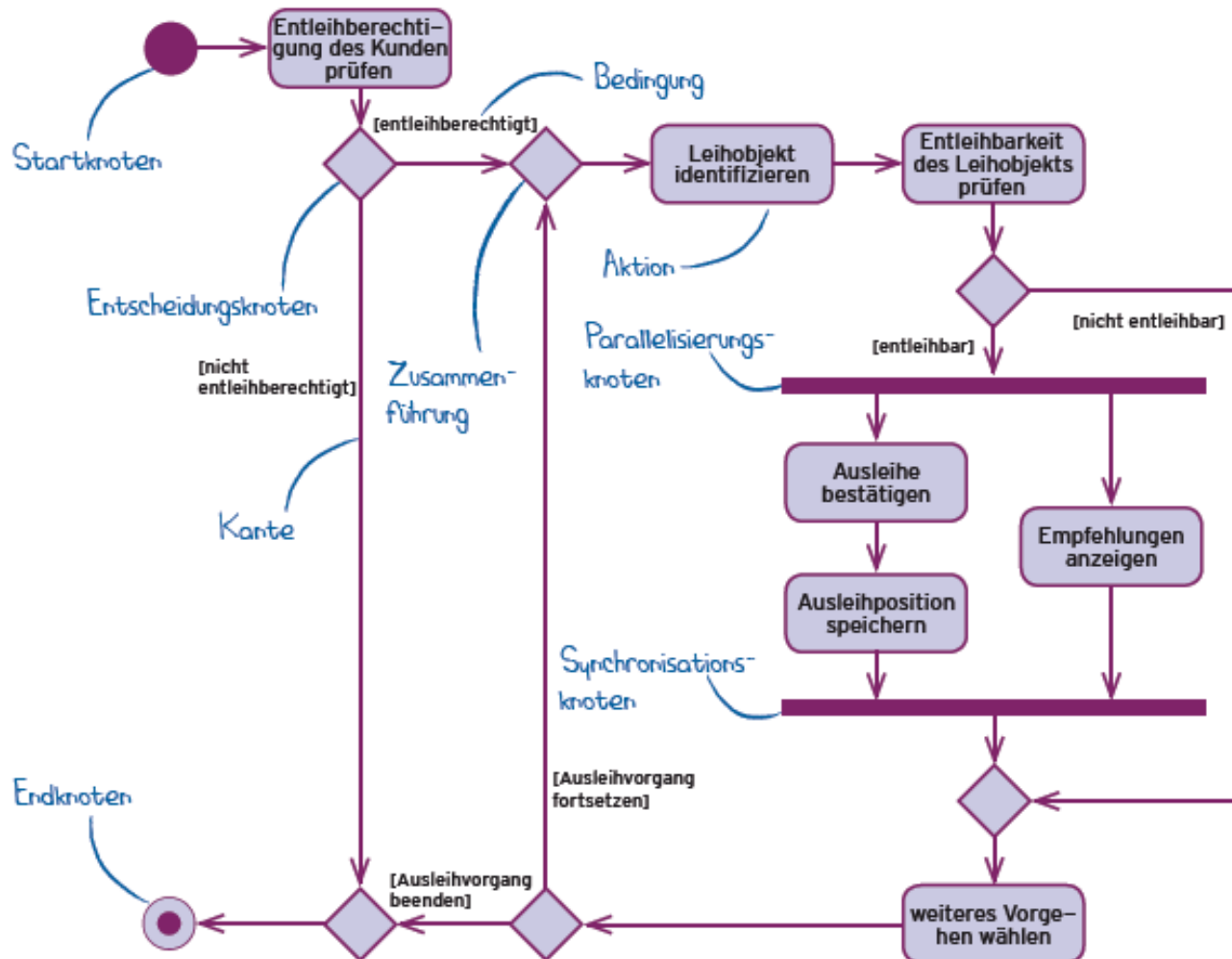
Eine Aktivität

- ist eine Menge von potentiellen Abläufen die sich in der Realität unter Randbedingungen abspielen

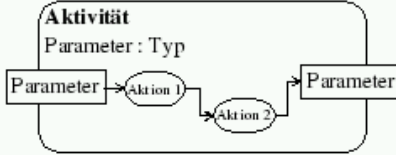
Ein Aktivitätsdiagramm

- zeigt den Rahmen und die Regeln von Verhaltensabläufen auf detailliertem Niveau
- zerlegt Aufgaben in Einzelschritte
- dient der Modellierung von Abläufen → es werden komplexe Verläufe unter Berücksichtigung von Nebenläufigkeiten sowie alternative Entscheidungswege im Detail dargestellt

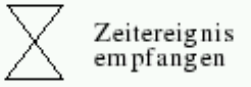
Beispiel Aktivitätsdiagramm



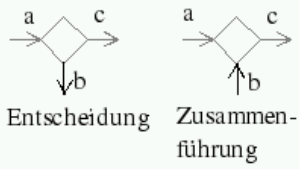
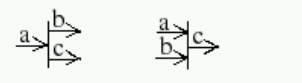
Notation Aktivitätsdiagramm

	Aktivität (Activity) Eine Aktivität besteht aus Knoten (Aktivitäten, Aktionen, Kontroll- und Objektknoten) und Kanten (Pfeilen), die den Kontrollfluss durch die Aktivität darstellen. Die Aktivität wird als abgerundetes Rechteck dargestellt. In der linken, oberen Ecke steht der Name der Aktivität, gefolgt vom Eingangsparameter und Typ. Die Rechtecke an beiden Seiten geben die Eingangs- bzw. Ausgangsparameter an. Die Symbole in der Aktivität stellen die Aktionen dar und die Pfeile den Kontrollfluss.
	Aktion (Action) Eine Aktion ist in UML das Basiselement zur Spezifikation von Verhalten. Eine Aktion kann als nicht weiter zerlegbare Funktion angesehen werden. Sie transformiert eine Eingabe zu einer Ausgabe. Aktionen sind in Aktivitäten enthalten. Die UML 2.0 unterscheidet verschiedene Typen von Aktionen.

Notation Aktivitätsdiagramm

 Signal senden Signal empfangen	Signal (Send signal action, Accept event action) Mit diesen Symbolen kann das Senden und Empfangen von Signalen bzw. Ereignissen dargestellt werden.
 Zeitereignis empfangen	Zeitereignis (Accept time event action) Über diese Symbol wird ausgedrückt, dass eine Aktion zeitgesteuert angestoßen wird.
	Kontrollknoten werden benutzt um Kontrollstrukturen in einem Flussgraph anzugeben.

Notation Aktivitätsdiagramm

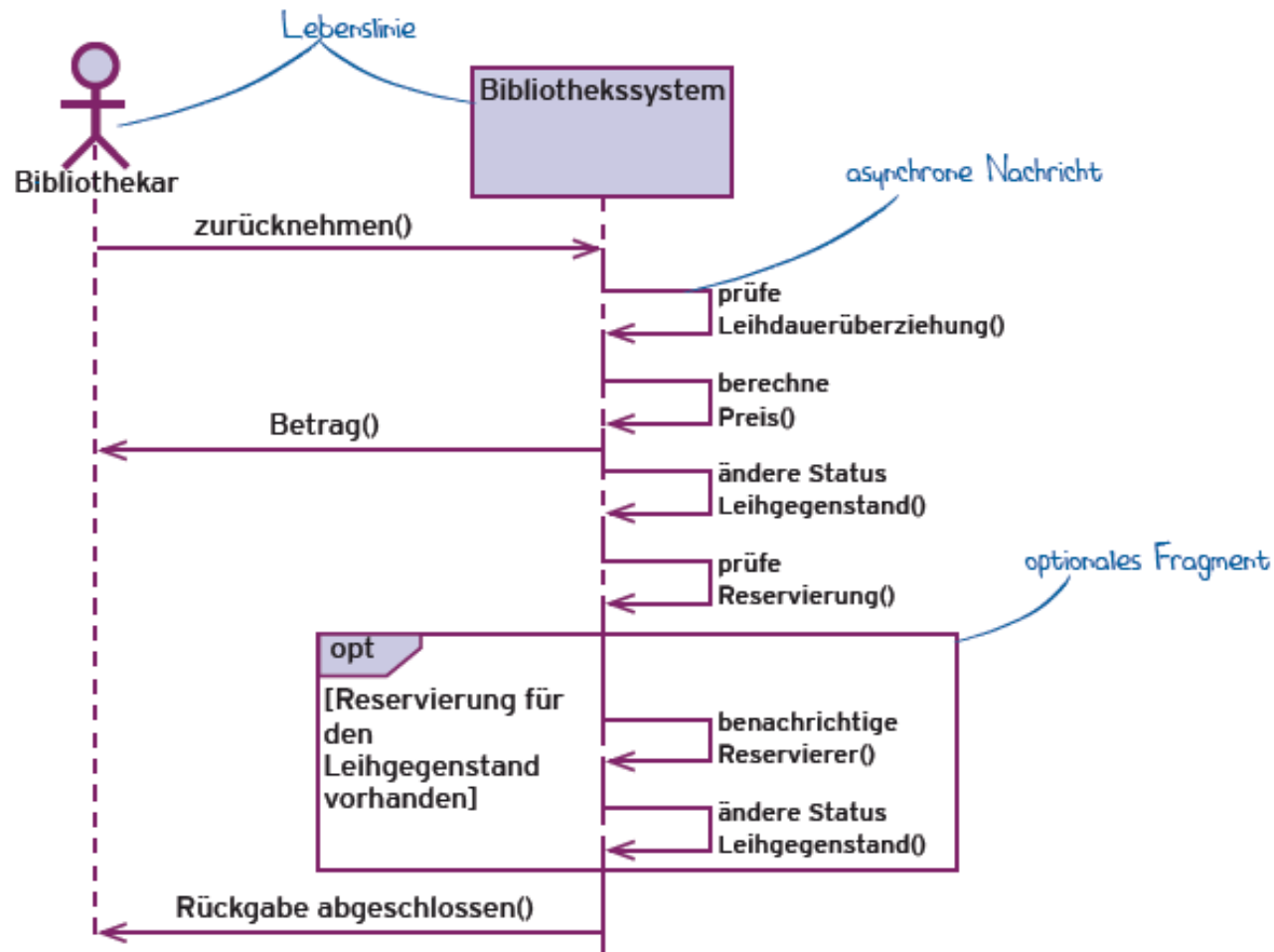
 Entscheidung Zusammenführung	Entscheidung und Zusammenführung Bei der Entscheidung wird nach Eintreffen des Tokens a entweder nach b oder nach c verzweigt. Die Bedingung, die erfüllt sein muss, kann als Ausdruck in eckigen Klammern an den Pfeilen angegeben werden. Bei der Zusammenführung wird nach Eintreffen der Token a oder b nach c verzweigt.
 Splitting Synchronisation	Splitting, Synchronisation Bei einem Splitting-Knoten wird in mehrere parallele Threads verzweigt (wenn a da ist, feuern b und c). Ein Synchronisationsknoten synchronisiert mehrere Threads (wenn a und b da sind, feuert c).
 Start-knoten End-knoten Ablaufende	Start-, Endknoten, Ablaufende Ein Startknoten (Initial Node) stellt den Beginn eines Ablaufes dar. Ein Endknoten (Activity Final Node) beendet eine Aktivität vollständig. Ein Ablaufende (Flow Final Node) terminiert einen Zweig einer Aktivität, die Aktivität selbst läuft weiter.

Sequenzdiagramm (vergleichbar mit einem Entscheidungsbaum)

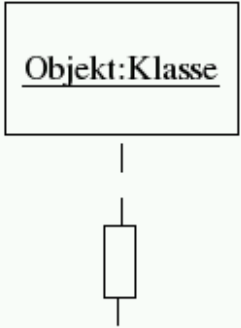
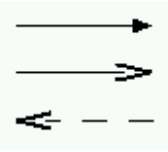
Wie läuft die Kommunikation in meinem System ab?

- zeigt den Informationsaustausch zwischen beliebigen Kommunikationspartnern (innerhalb eines Systems oder zwischen verschiedenen Systemen)
- ermöglicht die Modellierung von:
 - festen Reihenfolgen
 - zeitlichen und logischen Ablaufbedingungen
 - Schleifen
 - Nebenläufigkeiten
- zeigt Interaktion in zwei Dimensionen:
 - vertikal → zeitliche Abfolge der Kommunikationsschritte
 - horizontal → die miteinander kommunizierenden Partner

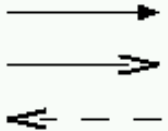
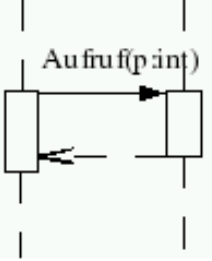
Beispiel Sequenzdiagramm



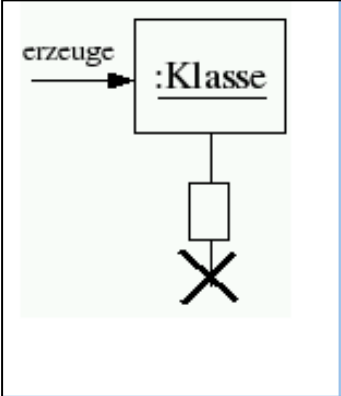
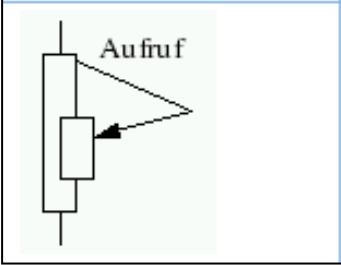
Notation Sequenzdiagramm

	Klasse, Objekt In dem Rechteck oberhalb der gestrichelten Linie wird der Objektname und der Klassenname angegeben. Der Name wird unterstrichen. Die senkrechte, gestrichelte Linie stellt die Lebenslinie (lifeline) eines Objekts dar. In diesem zeitlichen Bereich existiert das Objekt. Das schmale Rechteck auf der gestrichelten Linie stellt eine Aktivierung dar. Eine Aktivierung ist der Bereich, in dem eine Methode des Objektes aktiv ist (ausgeführt wird). Auf einer Lebenslinie können mehrere Aktivierungen enthalten sein.
	Nachricht, Botschaft Objekte kommunizieren über Nachrichten. Nachrichten werden als Pfeile zwischen den Aktivierungen eingezeichnet. Der Name der Nachricht steht an dem Pfeil. Eine Nachricht liegt immer zwischen einem sendenden und einem empfangenden Objekt.

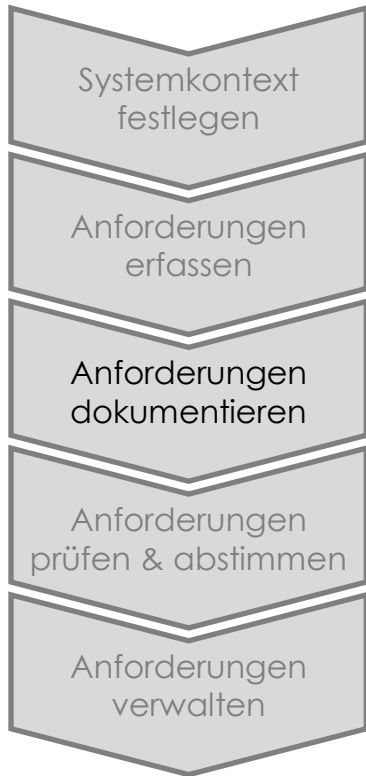
Notation Sequenzdiagramm

	Nachricht, Botschaft Objekte kommunizieren über Nachrichten. Nachrichten werden als Pfeile zwischen den Aktivierungen eingezeichnet. Der Name der Nachricht steht an dem Pfeil. Eine Nachricht liegt immer zwischen einem sendenden und einem empfangenden Objekt.
	Synchrone Nachricht (Aufruf) Der Pfeil mit der ausgefüllten Spitze bezeichnet einen synchronen Aufruf. Der Aufruf erfolgt von der Quelle zum Ziel, d. h. die Zielklasse muss eine entsprechende Methode implementieren. Die Quelle wartet mit der weiteren Verarbeitung bis die Zielklasse ihre Verarbeitung beendet hat und setzt die Verarbeitung dann fort. Ein Aufruf muss einen Namen haben; in runden Klammern können Aufrufparameter angegeben werden. Der gestrichelte Pfeil ist der Return. Die Bezeichnung des Return mit einem Namen ist optional.

Notation Sequenzdiagramm

	Objekt erzeugen, zerstören Das Erzeugen eines Objektes wird durch eine Nachricht, die im Kopf des Objekts endet dargestellt. Das Zerstören (Löschen) eines Objektes wird durch ein x auf der Lebenslinie markiert.
	Rekursion Sendet ein Objekt eine Nachricht an sich selbst, so ruft es eine Methode auf, die es selbst implementiert.

Anforderungsdokumentation – Styleguide



Styleguide festlegen

- Im Rahmen der Anforderungsanalyse ist ebenfalls ein einheitlicher Styleguide festzulegen
- Dient zur Sicherstellung eines einheitlichen und übergreifenden Layouts und Bedien- und Navigationskonzepts
- Wiederkehrende Elemente erhöhen die Akzeptanz beim Endanwender
- Inhalte sind alle Steuerelemente (Buttons, Radio-Buttons, Checkboxes, Icons, Filter, Tabellenaufbau, Ausgabe der Fehlermeldungen, Auswahl der Farbpalette, Shortcuts...)
- Dient als Vorgabe für das Maskendesign und -layout

Anforderungsdokumentation – Maskendesign

Systemkontext festlegen

Anforderungen erfassen

Anforderungen dokumentieren

Anforderungen prüfen & abstimmen

Anforderungen verwalten

A Web Page

http://IT-PM&SE.de

File Edit View Help

[Home](#) | [Hilfe](#) | [Handbuch](#)

Kundenname*

Strasse

PLZ

Ort

Neuanlage

Speichern

Löschen

Zurücksetzen

Suchen

//

ComboBox

Kundenname	Strasse	Nr.	PLZ	Ort
Kunde 1	Strasse 1	25	12345	Düsseldorf
Kunde 2	Strasse 2	28	54321	Köln

3

Anforderungsdokumentation – Maskendesign

Systemkontext festlegen

Anforderungen erfassen

Anforderungen dokumentieren

Anforderungen prüfen & abstimmen

Anforderungen verwalten

Tabreihenfolge:
1. Strasse
2. PLZ
3. Kundenname

Pflichtfeldvalidierung

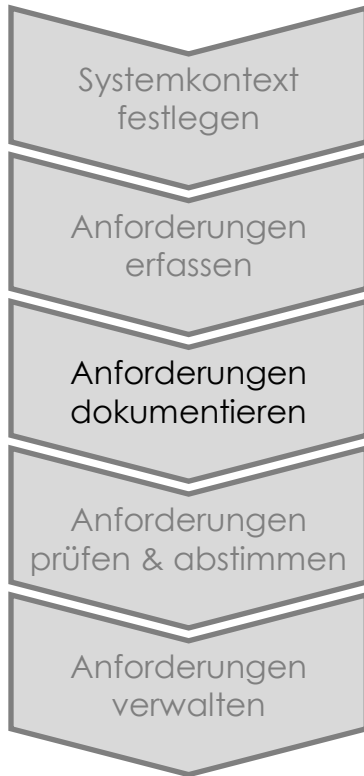
Enter-Taste: Default = Suche auslösen

Steht nur ausgewählten Nutzern zur Verfügung - Berechtigungskonzept

Button initial bei Maskenaufruf ausgegraut, darf erst aktiv sein, wenn vorher eine Suche ausgelöst wurde,

Kundenname	Strasse	Nr.	PLZ	Ort
Kunde 1	Strasse 1	25	12345	Düsseldorf
Kunde 2	Strasse 2	28	54321	Köln

Anforderungsdokumentation – Styleguide

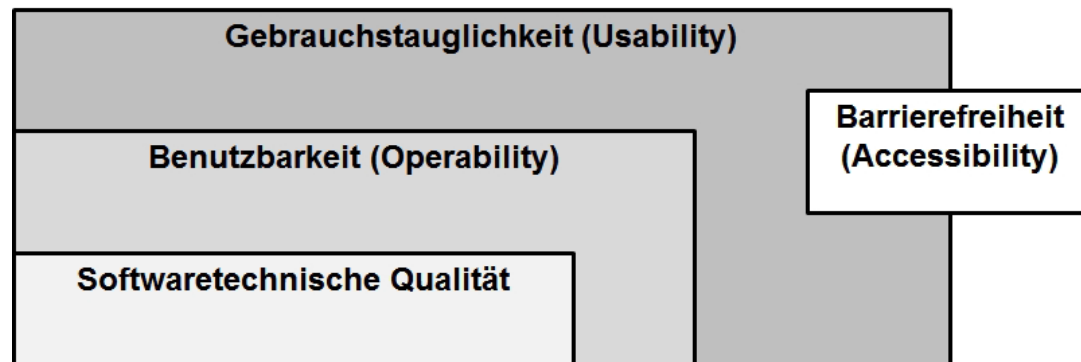


Navigationskonzept festlegen

- Ein Navigationskonzept sollte dem Anwender immer folgende vier Fragen beantworten:
 - Wo bin ich
 - Woher komme ich
 - Wohin kann ich gehen
 - Warum sollte ich da hingehen
- Im Wesentlichen geht es um die Frage wie gut sich der Anwender in der Applikation zurechtfindet
- Betrifft auch die Informationsarchitektur – wie werden welche Inhalte in den Masken dargestellt

Fragestellung nach der Benutzerfreundlichkeit zur effizienten Nutzung wirft folgende Qualitätsanforderungen auf

- Die Usability bezieht sich nicht nur auf die Benutzeroberfläche, sondern das gesamte System!
- Usability Evaluation unterscheidet
 - Summative Evaluation → wird am Ende einer App-Entwicklung durchgeführt, dient dem Vergleich mehrerer Produkte
 - Formative Evaluation → Analyse im Verlauf der Entwicklung mit dem Ziel Verbesserungen zu generieren oder Stabilität zu erhöhen



Fragestellung nach der Benutzerfreundlichkeit zur effizienten Nutzung wirft folgende Qualitätsanforderungen auf

Lernbarkeit (learnability)

- Beschreibt, wie leicht Aufgaben erledigt werden können, wenn der Benutzer eine Anwendung zum ersten Mal benutzt

Effizienz (efficiency)

- Gibt an, wie schnell Aufgaben durchgeführt werden können, wenn der Benutzer mit einer Anwendung vertraut ist

Einprägsamkeit (memorability)

- Erklärt, wie leicht sich Benutzer wieder in ein System einarbeiten können, wenn sie es für eine längere Zeitspanne nicht benutzt haben

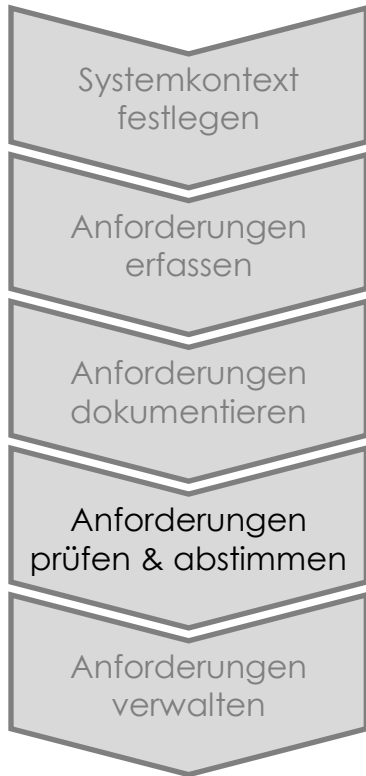
Fehler (errors)

- Beschreibt, wie viele Fehler ein Benutzer macht, wie gravierend diese für die Arbeit sind und wie schnell sie wieder ausgebessert werden können

Zufriedenheit (satisfaction)

- Ein Maß für die Freude, die ein Benutzer bei der Benutzung eines Produktes empfindet (joy of use)

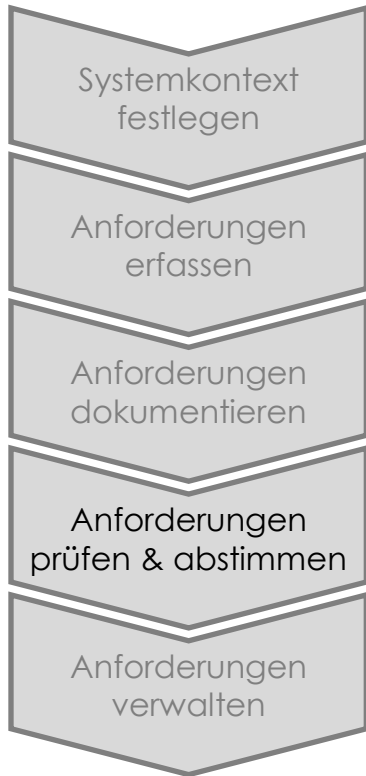
Projektvorbereitende und –begleitende QS der Anforderungen



Prinzipien der Prüfung von Anforderungen

- Beteiligung der richtigen Stakeholder/Prüfung aus unterschiedlichen Sichten
 - Anforderungssteller (fachseitig)
 - Requirements Engineer (Mediator Fachseite → Technik)
 - Entwicklungsabteilung (technisch)
 - Betrieb (technisch/infrastrukturell)
- Trennung Fehlersuche und Fehlerkorrektur
 - 1. Übergreifende Fehlersuche im Gesamtkontext aller Anforderungen → Aufdecken von Inkonsistenzen und Redundanzen
 - 2. Anschließende Korrektur der identifizierten Mängel durch Business Consulting/Architekten

Projektvorbereitende und –begleitende QS der Anforderungen

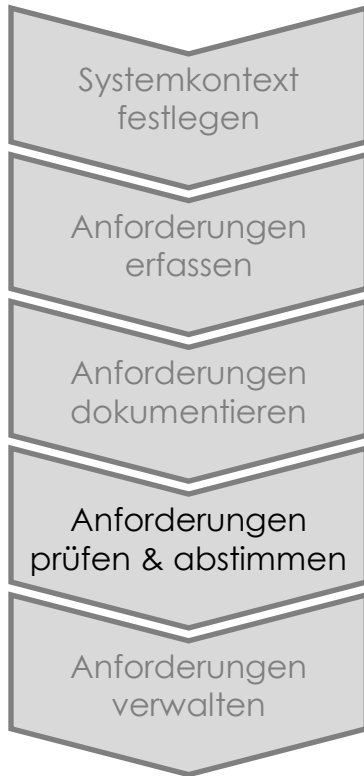


Prinzipien der Prüfung von Anforderungen

■ Dokumentationstiefe- und Form prüfen

- Wurden geeignete Methoden zur Darstellung der Anforderungen angewendet (z. B. EPK-Modellierung für Arbeitsabläufe, UML für Klassendiagramme)?
- Reicht der Detaillierungsgrad aus, um eine verbindliche Aufwandsschätzung durchzuführen, können geeignete Abnahmekriterien auf dieser Basis formuliert werden?
- Sind die vertraglich vereinbarten Dokumente (Anzahl und Form) erstellt und gemäß Delivery Modell geliefert worden?

Projektvorbereitende und –begleitende QS der Anforderungen

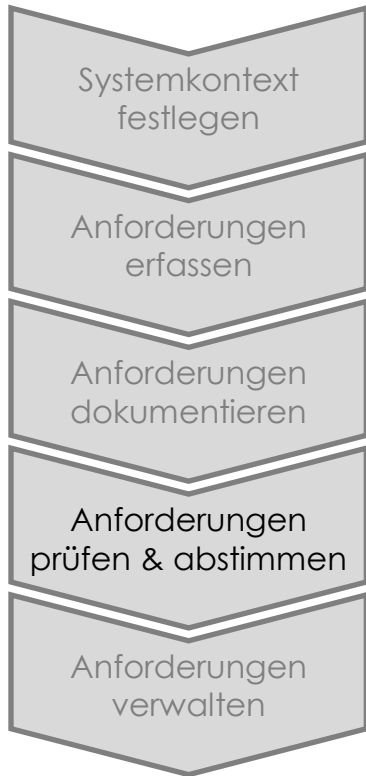


Prinzipien der Prüfung von Anforderungen

■ Wiederholte und kontinuierliche Prüfung

- Insbesondere bei langfristigen Projekten mit hoher Dynamik und instabilen Anforderungslage empfiehlt sich eine kontinuierliche entwicklungsbegleitende Prüfung
- Kann insb. bei Festpreisprojekten und/oder agilen Vorgehensmodellen hilfreich sein, um während der Realisierung Anhaltspunkte für Änderungsanforderungen zu identifizieren

Projektvorbereitende und –begleitende QS der Anforderungen

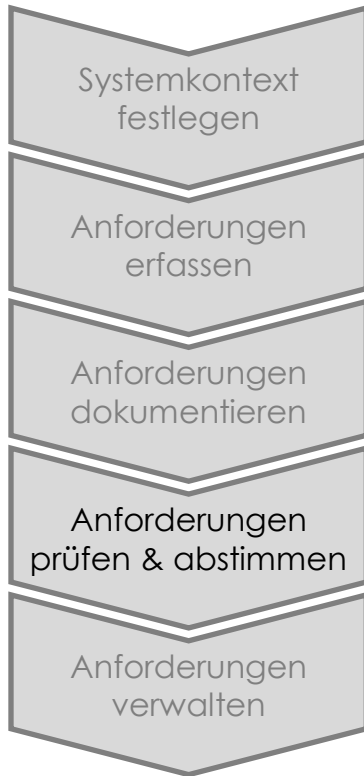


Techniken zur Prüfung von Anforderungen

■ Walkthrough

- Reviewrunde mit allen Beteiligten zur Prüfung der Anforderungen auf Korrektheit, Vollständigkeit, Eindeutigkeit... → nach der Reviewrunde sollten möglichst keine offenen Klärungspunkte mehr vorliegen
- Es werden alle erzeugten Dokumente geprüft und fachlich für die Realisierung freigegeben (Maskendesign, Prozessdiagramme...)

Projektvorbereitende und –begleitende QS der Anforderungen

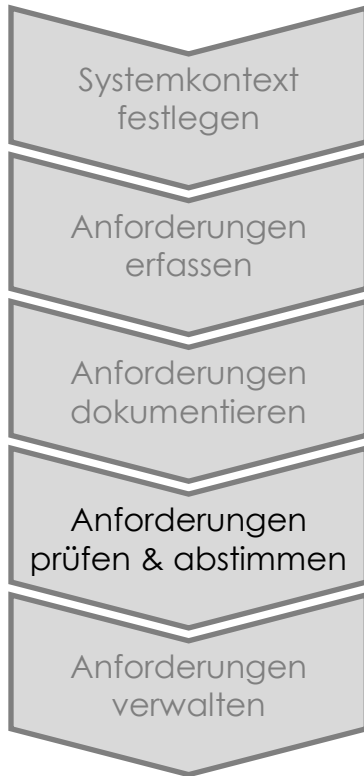


Techniken zur Prüfung von Anforderungen

■ Checkliste

- Ähnliches Verfahren wie die Walkthrough Methode. Auf Basis einer formal abgestimmten Checkliste werden die Punkte abgehakt und freigegeben
- Kann als Input für spätere Testfälle und -Szenarien dienen

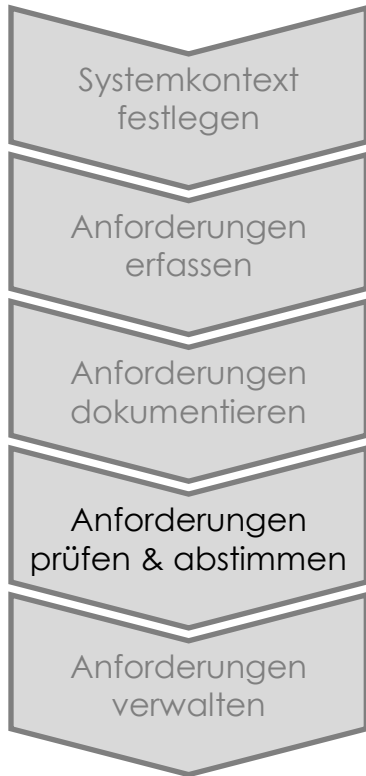
Projektvorbereitende und –begleitende QS der Anforderungen



Checkliste (exemplarisch)

Prüfung Maskendesign:	
nicht erfüllt	
Beschreibung der Prüfung:	
Tahoma 2 B I U	Festgestelltes Ergebnis:
Prüfen Sie das Design der Maske in der Anwendung mit dem Design in der Dokumentation. Sind die Felder/Oberflächenelemente angeordnet, wie im Design? Beachten Sie dass es aufgrund der Bildschirmgrößen durchaus zu Abweichungen in Größe und Abstände kommen kann. In folgenden Aspekten muss die Anwendung jedoch übereinstimmen:	
Sichtprüfung Look/Feel:	
nicht erfüllt	
Beschreibung der Prüfung:	
	Festgestelltes Ergebnis:
Entspricht die Maske auf dem ersten Blick dem allgemeinen Look&Feel der Anwendung?	
Erwartetes Ergebnis: Die Maske erscheint nicht ungewöhnlich im Vergleich zu anderen Anwendungsmasken im CCS.	
Prüfung Maskenfunktion:	
nicht erfüllt	
Beschreibung der Prüfung:	
	Festgestelltes Ergebnis:
Die in der Dokumentation beschriebenen Attribute der Maske dienen als Referenz zur Entscheidung, ob das erwartete Ergebnis eintritt oder eine Abweichung besteht. Hier ist besonders bei Eingabefeldern zu beachten, dass die Dokumentation die gültigen Werte beinhaltet (Integer vs. String usw). Nutzen Sie diese Angaben bei der Wahl der Testdaten und achten sie bei der Ausführung darauf möglichst auch ungültige Werte zu probieren.	
Erwartetes Ergebnis: Die Attribute der Maske sind in der Dokumentation spezifiziert.	

Projektvorbereitende und –begleitende QS der Anforderungen

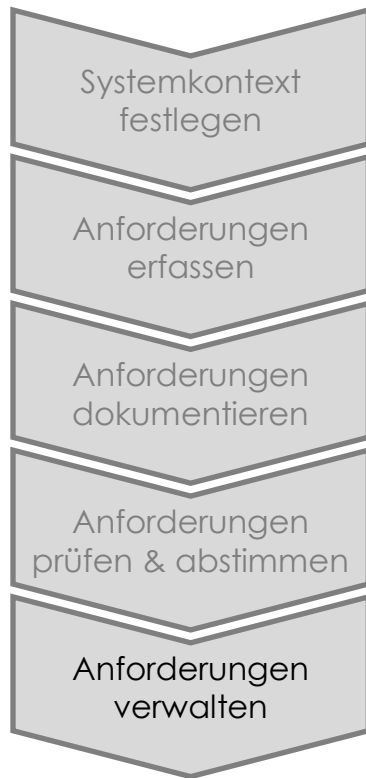


Techniken zur Prüfung von Anforderungen

■ Prototyping

- Wegwerf-Prototyp vs. evolutionärer Prototyp
- Wegwerf-Prototyp oftmals zu kost- und zeitintensiv (kritisch mit Blick auf „time-to-market“)
- Eignet sich ggf. für Teile der geplanten Anwendung, um ein Look-and-feel beim Endanwender zu erzeugen

Projektvorbereitende und –begleitende Verwaltung der Anforderungen

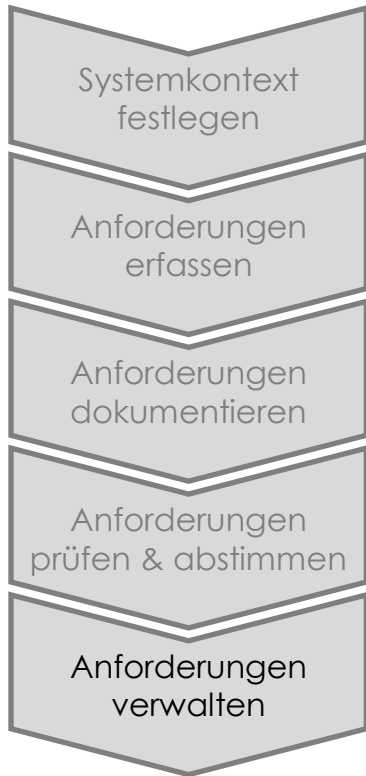


Priorisierung von Anforderungen

■ Priorisierungsmerkmale

- Kosten für die Umsetzung
- Risiko
- Wichtigkeit bzw. Schaden bei nicht erfolgter Umsetzung („must-have“ vs. „nice-to-have“)
- Volatilität (im Sinne der Stabilität)
- Zeitdauer für die Umsetzung

Projektvorbereitende und –begleitende Verwaltung der Anforderungen

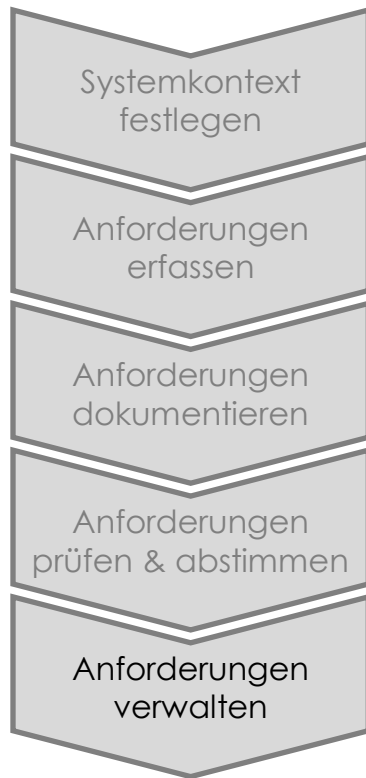


Priorisierungstechniken von Anforderungen

■ Klassifizierung

- Mandatory (Unbedingt erforderlich, andernfalls ist der Erfolg des Systems gefährdet)
- Optional (teilweises vernachlässigen von Anforderungen dieser Kategorie gefährden den Erfolg nicht)
- Nice-to-have (Eine Nichtberücksichtigung gefährdet den Erfolg des Projekts nicht → „nice-to-haves“ sind nicht mit Change Requests zu verwechseln)

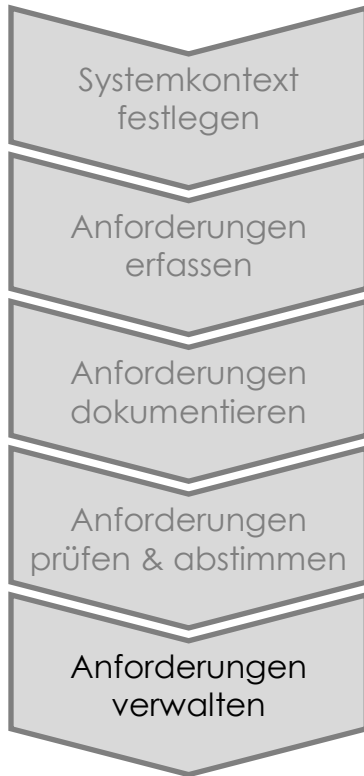
Projektvorbereitende und –begleitende Verwaltung der Anforderungen



Priorisierungstechniken von Anforderungen

- Klassifizierung nach dem Kano Modell
 - Basismerkmal
 - Leistungsmerkmal
 - Begeisterungsmerkmal

Projektvorbereitende und -begleitende Verwaltung der Anforderungen

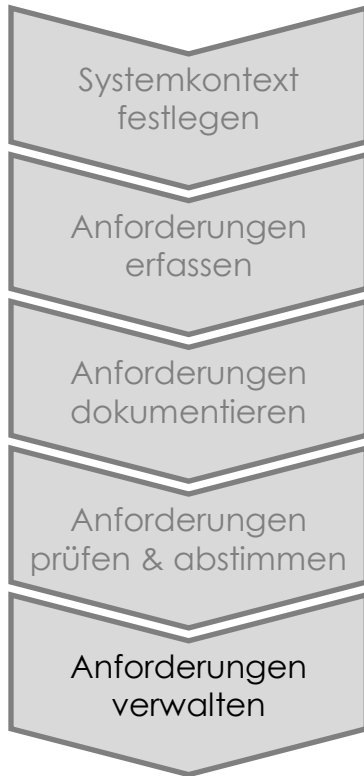


Priorisierungstechniken von Anforderungen

■ Wiegers'sche Priorisierungsmatrix

Relatives Gewicht	1 → 2 (GewichtNutzen)	→ 1 (GewichtNachteil)			→ 1 (GewichtKosten)		→ 0,5 (GewichtRisiko)			
Anforderung	Relativer Nutzen	Relativer Nachteil	Gesamt	Wert %	Relative Kosten	Kosten %	Relatives Risiko	Risiko %	Priorität	Rang
2 R ₁	5	3	13	16,8	2	13,3	1	9,1	0,941	1
R ₂	9	7	25	32,5	5	33,3	3	27,2	0,692	3
R ₃	5	7	17	22,1	3	20,0	2	18,2	0,759	2
R ₄	2	1	5	6,5	1	6,7	1	9,1	0,577	4
↓ R ₅	4	9	17	22,1	4	26,7	4	36,4	0,489	5
Gesamt	25	27	77	100	15	100	11	100		
	3	4	5		6		7		8	9

Projektvorbereitende und –begleitende Verwaltung der Anforderungen



Priorisierungstechniken von Anforderungen

■ Wiegers'sche Priorisierungsmatrix

1. Bestimmung der relativen Gewichte von Nutzen, Nachteil, Kosten und Risiko
2. Bestimmung der zu priorisierenden Anforderungen
3. Abschätzung des relativen Nutzens
4. Abschätzung des relativen Nachteils
5. Berechnung der Gesamtwerte und der % Werte pro Anforderung
6. Abschätzung der relativen Kosten und Berechnung der % Kosten pro Anforderung
7. Abschätzung des relativen Risikos und Berechnung des % Risikos pro Anforderung
8. Berechnung der einzelnen Anforderungsprioritäten
9. Festlegung des Rangs der einzelnen Anforderungen

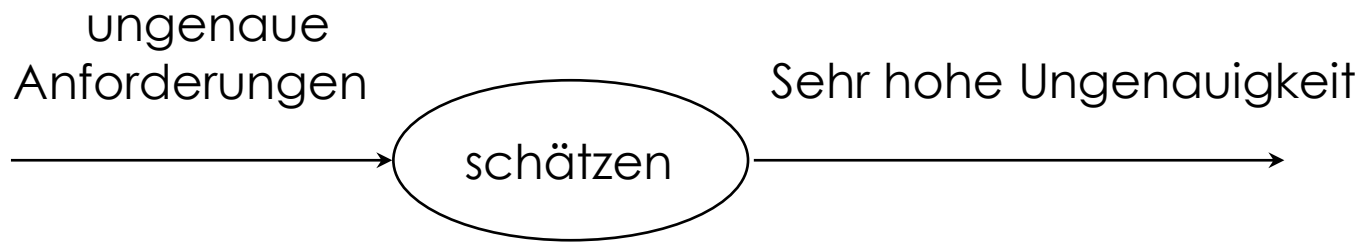
Aufwandsschätzung – Ausgangslage

- Optimale Projekte mit "Punktlandungen", in Bezug auf zu Projektanfang abgegebene Schätzwerte, sind äußerst selten bzw. gar nicht anzutreffen
- Analysiert man einmal solch seltene "Punktlandungen", so wird man in den meisten Fällen feststellen, dass es sich um sehr pessimistische Schätzungen handelt, die mit sehr hohen Sicherheitszuschlägen ausgestattet sind
- Diese nach außen hin sehr gut aussehenden Projekte sind im Grunde aber nicht sehr wirtschaftlich, da die von Anfang des Projekts viel zu hoch angesetzten Personal-, Zeit- und Kosten-Etats generell auch ausgeschöpft werden
- Schätzungen sind meist zu optimistisch

Aufwandsschätzung – Probleme in der Praxis

- Schätzungen sind ungenau
- Dilemma: je früher eine Schätzung durchgeführt wird, desto ungenauer ist das Ergebnis, dennoch ist eine frühe Aufwandsschätzung erforderlich, um die Zeit- und Kosten-Dimension eines Projekts zu umreißen und die Ressourcenplanung vorzubereiten
- Wiederholungen im weiteren Projektverlauf sind zur Präzisierung notwendig → z. B. kann das Budget eingehalten werden?

Aufwandsschätzung – Probleme und ihre Folgen



- Prinzip: Garbage in garbage out
- Ungenau spezifizierte Anforderungen erhöhen das Risiko einer ungenauen Schätzung und damit das Scheitern eines Projekts direkt zu Beginn
- Aussage über erforderliche Budgethöhe und Zeitdauer des Projekts nicht möglich, dadurch droht auch ein kommerzieller Schaden

Aufwandsschätzung – Häufigste Fehler

- Neue Themen werden oft zu niedrig eingeschätzt
- Mitarbeiter, die noch nie Schätzungen vorgenommen haben, schätzen den Aufwand eher zu gering ein
- Aus Zeitmangel wird oft eine bewusste und systematische Erfahrungssicherung nicht durchgeführt. Somit können weitere Schätzungen nicht von der Erfahrung profitieren
- Unter dem Druck knapper Ressourcen und enger Terminpläne werden viele Angaben zum voraussichtlichen Aufwand zu knapp geschätzt – und sind deswegen von vornherein unrealistisch
- Nicht immer werden die zuständigen Mitarbeiter bei der Aufwandschätzung mit eingebunden
- Der Aufwand für Projektmanagement → Controlling und Qualitätssicherung wird nicht berücksichtigt!

Aufwandsschätzung – Verfahren

Es gibt eine Reihe von Aufwandsschätzmethoden. Im wesentlichen werden drei Klassen unterschieden:

1. Expertenschätzung

- Delphi Methode
- Zwei-Punkt-Schätzung/Drei-Punkt-Schätzung

2. Parametrische Kostenschätzungen

- Function-Point-Methode
- COCOMO (Constructive Cost Model)

3. Kostenschätzung mit Kostenkennziffern und Kennziffernsystemen

Aufwandsschätzung – Expertenschätzungen

Einzelschätzung ▪ Ein einziger Experte legt die Schätzwerte für einen bestimmten Aufwandsposten fest ▪ Genauigkeit hängt wesentlich von der Erfahrung des Experten ab ▪ Hohe Verantwortung für eine Person ▪ Einseitige Beurteilung von Aufwänden und Unsicherheiten	Delphi-Methode ▪ Mehrere Experten führen ihre Schätzung anonym und ohne Abstimmung untereinander durch ▪ Zusammenführung der Schätzung durch den Projektleiter ggf. in Iterationen bei (größeren) Abweichungen ▪ Korrektur-Möglichkeit der Schätzzahl ohne „Gesichtsverlust“	Schätzklausur ▪ Mehrere Experten schätzen im Rahmen eines gemeinsamen Workshops ▪ Sofortige Zusammenführung von Aufwänden und Plausibilisierung ▪ Inhaltliche Diskussion bei großen Abweichungen ▪ Risiken werden bewusst ▪ Gleichmäßiger Informationsstand im Team
Pragmatisch aber leicht ungenau	Verlässlich, aber sehr zeitaufwändig	Besser als Mittelwerte, ebenfalls zeitaufwändig

Aufwandsschätzung – Delphi Methode

Delphi Methode

- Die Aufwandsschätzung erfolgt im Rahmen einer Schätzklausur unter Anleitung eines Moderators:

Prinzipieller Ablauf

- Moderator erläutert die Aufgabenstellung
- Diskussion der Aufgabenstellung durch die Experten
- Getrennte Schätzung durch die Experten
- Zusammenfassung der Schätzwerte und Dokumentation
- Diskussion der Ergebnisse und Abweichungen
- Schätzungen überarbeiten
- Iteration bis eine ausreichende Annäherung der Schätzungen erreicht ist

Aufwandsschätzung – Zwei-Punkt-Schätzung (Expertenschätzung)

Zwei-Punkt-Schätzung

Problem:

- Ist eine hinreichende Zerlegung in Teilprobleme nicht möglich, ist es schwer gute Schätzwerte mit hinreichender Genauigkeit zu finden.

Die Experten werden nach folgenden Werten befragt:

- Best case: Idealfall, niedriger Wert
- Worst case: Schlechterster Fall, höchste Werte

Aus diesen Werten wird der Schätzwert nach folgender Formel berechnet

$$\text{Schätzwert} = \frac{(\text{Best Case} + \text{Worst case})}{2}$$

Aufwandsschätzung – Drei-Punkt-Schätzung (Expertenschätzung)

Drei-Punkt-Schätzung

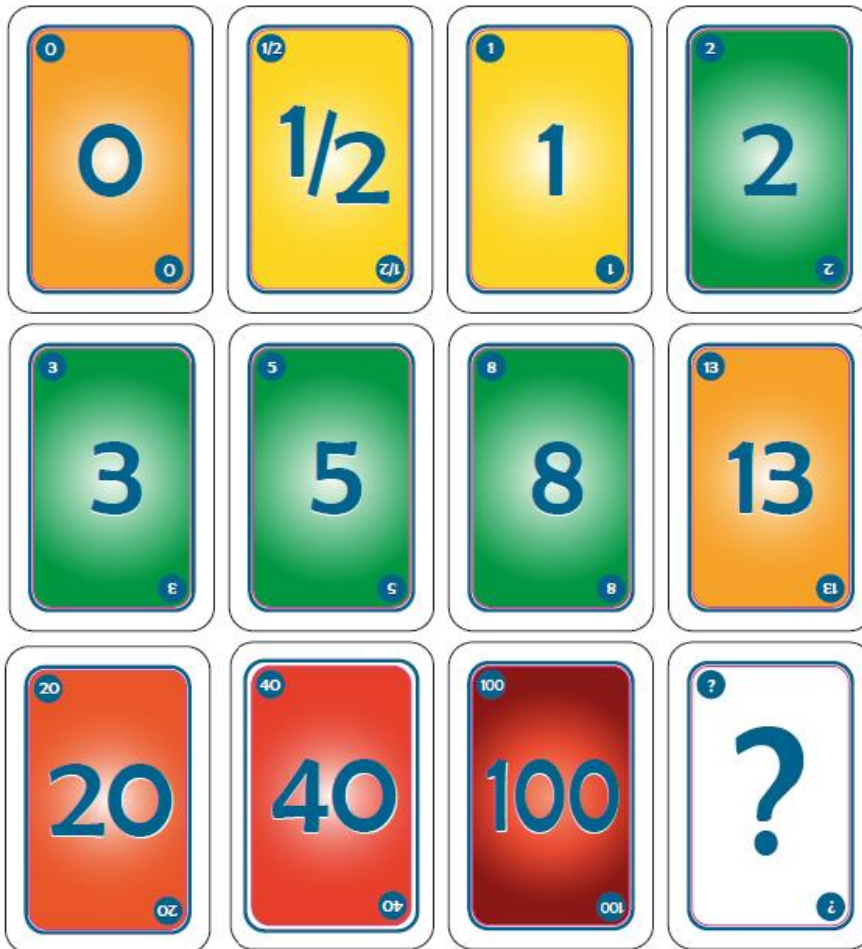
Die Experten werden nach folgenden Werten befragt:

- Optimistisch: Idealfall, niedriger Wert
- Realistische: Wahrscheinlichster Fall, mittlerer Wert
- Pessimistisch: Schlechtesten Fall, höchste Werte

Aus diesen Werten wird der Schätzwert nach folgender Formel berechnet

$$\text{Schätzwert} = \frac{\text{Opt} + 4 \times \text{Real} + \text{Pess}}{6}$$

Aufwandsschätzung – Schätzpoker (agile Projekte/SCRUM)



Quelle: Wordpress.com

Qualitätsmanagement beginnt während der Realisierung

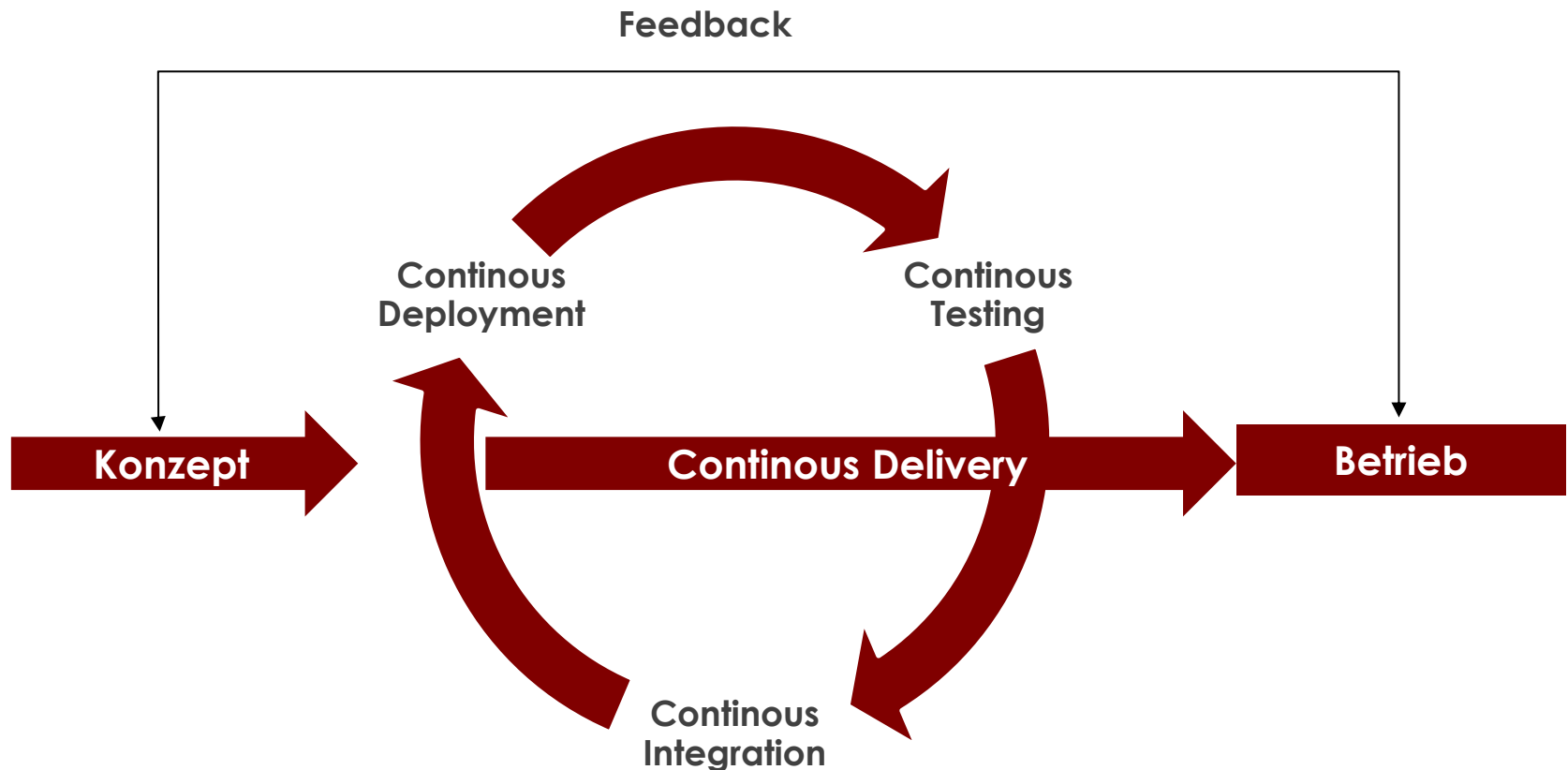
Codierrichtlinien festlegen

- Einheitliche Codierrichtlinien vermeiden einen „schlechten Code“
- Codeprüfung und Validierung bei Check-In des Codes etablieren (systemgestützt und/oder 4-Augen Prinzip mit leitendem Architekten)
- Sauberer und gut kommentierter Code erhöht Wartbarkeit und erleichtert die Einarbeitung möglicher Nachfolger

Kontinuierliche Integration einführen

- **Nightly Build-Prozess**
 - nächtliche Aufbereitung der eingecheckten Codeteile (zeitgesteuert) und
 - ggf. automatisierte Installation auf einer Integrations-/Testumgebung
- **Continuous Integration**
 - analog zu Nightly Build (allerdings eventgesteuert), zusätzlich werden automatisierte Tests ausgeführt
 - Versionsverwaltung
- **Continuous Delivery**
 - Kontinuierliche Auslieferung einer produktionsreifen Software

Qualitätsmanagement beginnt während der Realisierung



Qualitätssicherung während der Entwicklung

Testautomatisierung

- Fachlich getriebene (automatisierte) Tests, die den Aufwand für Regressionstests deutlich reduzieren
- Setzen in der Regel ein definiertes Testset und stabile bzw. wiederherstellbare Testdaten voraus
- Werden häufig im Nightly Build/Continuous Integration Prozess integriert und nach erfolgreichem Build-Prozess ausgeführt

Unit Tests

- Sog. Modultests, die während der Entwicklung geschrieben werden
- Unit Tests prüfen auf technischem Weg, ob die Permutationen einer Methode zum richtigen Ziel führen
- Werden zur Prüfung der Geschäftslogikebene erstellt

Qualitätssicherung während der Entwicklung

Last- und Performancetests

- Simulieren ein Lastverhalten unter Echtbedingungen auf den Test- und/oder späteren Produktionsservern
- Prüft technisch, ob ein System „gesund“ skaliert und eine erwartungskonforme Lastverteilung durchführt
- Gibt Aufschluss, ob ein Memory Leak, eine hohe CPU-Auslastung oder ein Überlauf des Caches bei entsprechender Last vorliegt

Integrationstest

- Beweist das Zusammenspiel einzelner Komponenten (auch mit internen/externen Umsystemen)
- Umfassen i.d.R. auch Funktions- und Schnittstellentests

Motivation/Grundlagen

- IT-Unternehmensarchitektur
- Einordnung in den Kontext / Zusammenspiel strategisch - operativ
- Herausforderungen gewachsener IT-Systemlandschaften
- Klassifizierung von Business Applikationen / Bestimmung des Applikationsportfolios
- *IT-Investitionsentscheidungen (ab Folie 307)*